



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №4

«Розв'язання нелінійних рівнянь на комп'ютері»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Порєв В. М.*

Тема: «Розв’язання нелінійних рівнянь на комп’ютері».

Мета: Метою даного заняття є ознайомлення з методиками та вивчення різних алгоритмів розв’язання нелінійних рівнянь на комп’ютері.

Хід роботи

У даній лабораторній роботі розглядається задача чисельного розв’язання нелінійного рівняння методом половинного ділення. Для виконання роботи було обрано рівняння

$$2^x - 4x = 0.$$

Введемо функцію

$$f(x) = 2^x - 4x.$$

Необхідно побудувати графік функції, відокремити корені на заданому проміжку, уточнити їх із заданою точністю ε , а також реалізувати програму з графічним інтерфейсом для виконання цих дій.

Порядок виконання роботи:

- Ознайомитись із методикою розв’язання нелінійних рівнянь на ПК.
- Побудувати графік функції лівої частини рівняння.
- Відокремити корені рівняння на заданому проміжку.
- Вибрати значення точності обчислень ε .
- На кожному проміжку уточнити корені методом половинного ділення.
- Початкові дані та результати обчислень вивести в інтерфейсі програми.
- Оформити звіт із описом методу, результатами та програмним кодом.

Початкові дані варіанта:

Рівняння	Метод	Проміжок пошуку	Очікувані корені
$2^x - 4x = 0$	половинного ділення	$[0; 4]$	0.310; 4.0

Для тестового запуску програми було використано такі параметри:

$$a = 0, \quad b = 4, \quad h = 0.1, \quad \varepsilon = 10^{-5}, \quad N_{\max} = 100.$$

Тут h — крок сканування проміжку для відокремлення коренів, а $N_{\max}$ — максимальна кількість ітерацій уточнення на одному проміжку.

Схема алгоритму:

1. Задати функцію $f(x) = 2^x - 4x$, межі a, b , крок h , точність ε та максимальну кількість ітерацій.
2. Перевірити коректність вхідних даних: $b > a$, $h > 0$, $\varepsilon > 0$.
3. Просканувати проміжок $[a; b]$ із кроком h .

4. Для кожної пари сусідніх точок перевірити зміну знака функції.
5. Сформувати список проміжків, на яких потрібно уточнити корінь.
6. Для кожного проміжку виконати метод половинного ділення.
7. На кожній ітерації обчислити середину проміжку та значення функції в ній.
8. Замінити одну з меж проміжку так, щоб корінь залишився всередині нового проміжку.
9. Завершити уточнення, коли досягнуто точності ε .
10. Вивести таблицю результатів, побудувати графік та сформувати текстовий звіт.

Результати обчислень:

Для першого кореня, відокремленого на проміжку $[0.3; 0.4]$, отримано таку ітераційну таблицю.

k	a_k	b_k	c_k	$f(c_k)$	Δ_k
1	0.3000000000	0.4000000000	0.3500000000	$-1.254394 \cdot 10^{-1}$	$5.000000 \cdot 10^{-2}$
2	0.3000000000	0.3500000000	0.3250000000	$-4.733556 \cdot 10^{-2}$	$2.500000 \cdot 10^{-2}$
3	0.3000000000	0.3250000000	0.3125000000	$-8.142188 \cdot 10^{-3}$	$1.250000 \cdot 10^{-2}$
4	0.3000000000	0.3125000000	0.3062500000	$1.148951 \cdot 10^{-2}$	$6.250000 \cdot 10^{-3}$
5	0.3062500000	0.3125000000	0.3093750000	$1.670754 \cdot 10^{-3}$	$3.125000 \cdot 10^{-3}$
6	0.3093750000	0.3125000000	0.3109375000	$-3.236445 \cdot 10^{-3}$	$1.562500 \cdot 10^{-3}$
7	0.3093750000	0.3109375000	0.3101562500	$-7.830272 \cdot 10^{-4}$	$7.812500 \cdot 10^{-4}$
8	0.3093750000	0.3101562500	0.3097656250	$4.438179 \cdot 10^{-4}$	$3.906250 \cdot 10^{-4}$
9	0.3097656250	0.3101562500	0.3099609375	$-1.696160 \cdot 10^{-4}$	$1.953125 \cdot 10^{-4}$
10	0.3097656250	0.3099609375	0.3098632813	$1.370981 \cdot 10^{-4}$	$9.765625 \cdot 10^{-5}$
11	0.3098632813	0.3099609375	0.3099121094	$-1.625967 \cdot 10^{-5}$	$4.882813 \cdot 10^{-5}$
12	0.3098632813	0.3099121094	0.3098876953	$6.041904 \cdot 10^{-5}$	$2.441406 \cdot 10^{-5}$
13	0.3098876953	0.3099121094	0.3098999023	$2.207964 \cdot 10^{-5}$	$1.220703 \cdot 10^{-5}$
14	0.3098999023	0.3099121094	0.3099060059	$2.909977 \cdot 10^{-6}$	$6.103516 \cdot 10^{-6}$

Оскільки на 14-й ітерації виконано умову

$$|f(c_{14})| = 2.909977 \cdot 10^{-6} \leq 10^{-5},$$

уточнення першого кореня завершується. Другий корінь дорівнює 4.0000000000, оскільки

$$f(4) = 2^4 - 4 \cdot 4 = 16 - 16 = 0.$$

Підсумкова таблиця результатів:

№	Проміжок	Корінь	$f(x)$	Кількість ітерацій
1	$[0.3000000000; 0.4000000000]$	0.3099060059	$2.909977 \cdot 10^{-6}$	14
2	$x = 4.0000000000$	4.0000000000	0	0

Блок-схема алгоритму:

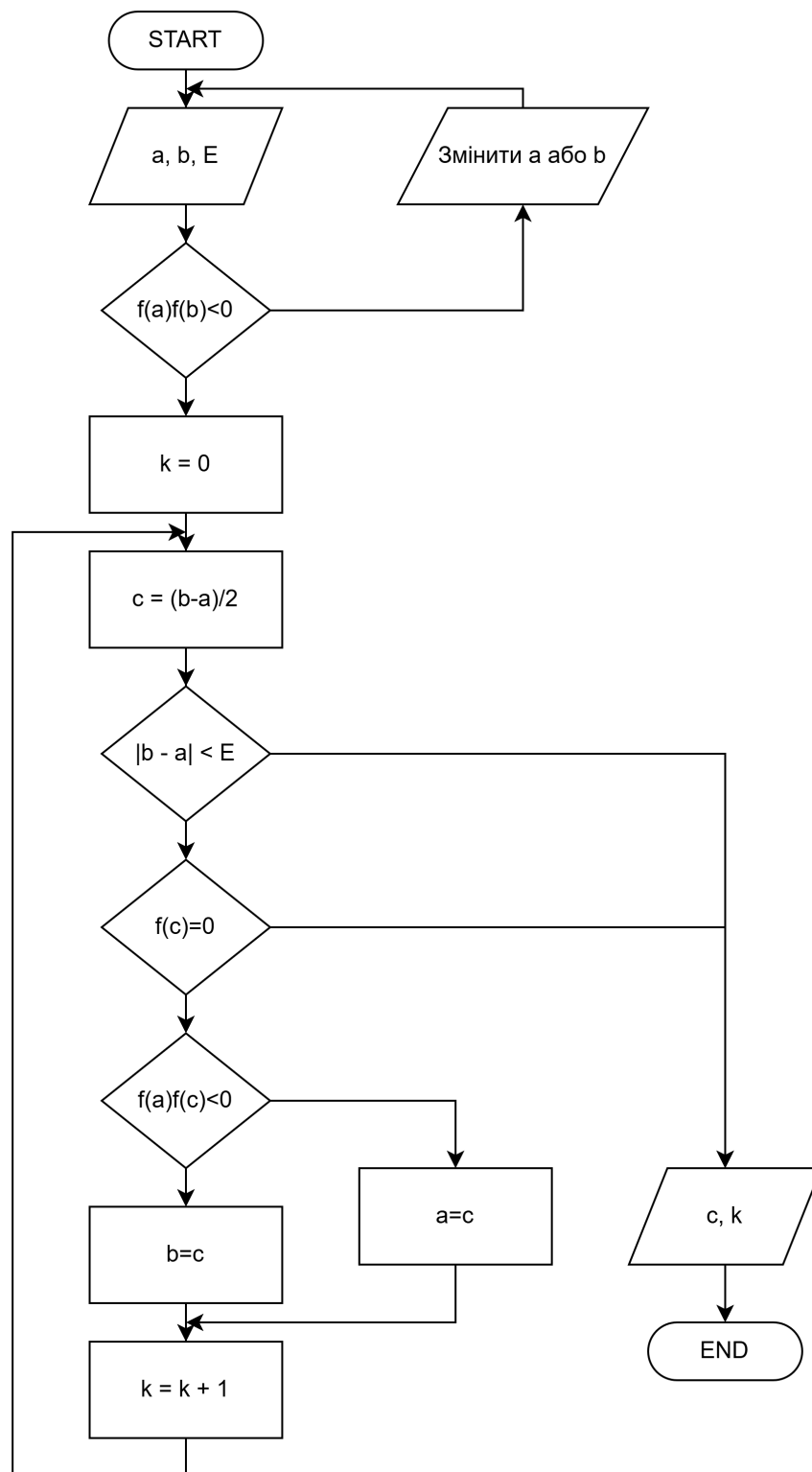


Рис. 1: Блок-схема алгоритму методу половинного ділення

Опис програмної реалізації:

Програма написана мовою Python із розділенням відповідальностей між модулями. Такий підхід відповідає базовим інженерним принципам SWEBOOK: модульність, зрозумілі межі відповідальності, перевірка вхідних даних, відокремлення алгоритмічної частини від графічного інтерфейсу.

Фактична структура проекту:

```
project/  
|-- app.py  
|-- src/  
|   |-- bisection.py  
|   |-- error_analysis.py  
|   |-- functions.py  
|   |-- graph_build.py  
|   |-- gui.py  
|   |-- io_utils.py  
|   |-- models.py  
|   |-- report.py  
|   |-- theme.py  
|-- test_json/  
    |-- default.json  
    |-- high_precision.json  
    |-- wide_interval.json
```

`app.py` відповідає за запуск програми. Модуль `functions.py` містить цільову функцію. Модуль `bisection.py` реалізує відокремлення коренів та метод половинного ділення. Модулі `models.py`, `error_analysis.py`, `report.py` та `io_utils.py` відповідають за структури даних, підготовку таблиць, формування звіту та роботу з JSON-файлами. Модуль `graph_build.py` готує дані для побудови графіка, а `gui.py` реалізує графічний інтерфейс користувача.

У графічному інтерфейсі передбачено:

- введення меж проміжку, кроку, точності та кількості точок графіка;
- завантаження параметрів із JSON-файлу;
- запуск обчислень;
- виведення таблиці коренів;
- збереження текстового звіту;
- побудову інтерактивного графіка з можливістю перетягування та масштабування.

Програмний код:

(project/app.py)

```
1 from src.gui import main  
2  
3  
4 if __name__ == "__main__":  
5     main()
```

(project/src/models.py)

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4
5
6 @dataclass(frozen=True)
7 class PageConfig:
8     page_id: str
9     nav_title: str
10    title: str
11    subtitle: str
12
13
14 @dataclass(frozen=True)
15 class RootInterval:
16     left: float
17     right: float
18
19     @property
20     def width(self) -> float:
21         return abs(self.right - self.left)
22
23     def as_text(self) -> str:
24         if self.width == 0:
25             return f"x = {self.left:.10f}"
26             return f"[{self.left:.10f}; {self.right:.10f}]"
27
28
29 @dataclass(frozen=True)
30 class IterationRow:
31     iteration: int
32     left: float
33     right: float
34     midpoint: float
35     f_left: float
36     f_midpoint: float
37     interval_length: float
38     estimated_error: float
39
40
41 @dataclass(frozen=True)
42 class RootResult:
43     interval: RootInterval
44     root: float
45     function_value: float
46     iterations: int
47     estimated_error: float
48     history: list[IterationRow] = field(default_factory=list)
49
50
51 @dataclass(frozen=True)
52 class ErrorRow:
53     index: int
```

```

54     interval: RootInterval
55     root: float
56     function_value: float
57     estimated_error: float
58     iterations: int

```

(project/src/functions.py)

```

1  from __future__ import annotations
2
3  import math
4  from collections.abc import Callable
5
6
7  Function = Callable[[float], float]
8
9
10 def target_function(x_value: float) -> float:
11     return math.pow(2.0, x_value) - 4.0 * x_value
12
13
14 def get_function_by_name(function_name: str) -> tuple[Function, str]:
15     functions: dict[str, tuple[Function, str]] = {
16         "target": (target_function, "2^x - 4x = 0"),
17     }
18
19     try:
20         return functions[function_name]
21     except KeyError as exc:
22         raise ValueError(f"Невідома функція: {function_name}") from exc

```

(project/src/bisection.py)

```

1  from __future__ import annotations
2
3  from collections.abc import Callable
4
5  from src.models import IterationRow, RootInterval, RootResult
6
7
8  Function = Callable[[float], float]
9
10
11 def isolate_root_intervals(
12     func: Function,
13     left: float,
14     right: float,
15     scan_step: float,
16     zero_tolerance: float = 1e-12,
17 ) -> list[RootInterval]:
18     if right <= left:
19         raise ValueError("Права межа має бути більшою за ліву.")

```

```

20     if scan_step <= 0:
21         raise ValueError("Крок відокремлення коренів має бути додатним.")
22
23     intervals: list[RootInterval] = []
24     x_left = left
25     f_left = func(x_left)
26
27     if abs(f_left) <= zero_tolerance:
28         intervals.append(RootInterval(x_left, x_left))
29
30     while x_left < right:
31         x_right = min(x_left + scan_step, right)
32         f_right = func(x_right)
33
34         if abs(f_right) <= zero_tolerance:
35             intervals.append(RootInterval(x_right, x_right))
36         elif f_left * f_right < 0:
37             intervals.append(RootInterval(x_left, x_right))
38
39         x_left = x_right
40         f_left = f_right
41
42     return _deduplicate_intervals(intervals, zero_tolerance)
43
44
45 def solve_bisection(
46     func: Function,
47     interval: RootInterval,
48     epsilon: float,
49     max_iterations: int,
50 ) -> RootResult:
51     if epsilon <= 0:
52         raise ValueError("Точність epsilon має бути додатною.")
53     if max_iterations < 1:
54         raise ValueError("Максимальна кількість ітерацій має бути не меншою за 1.")
55
56     left = interval.left
57     right = interval.right
58     f_left = func(left)
59     f_right = func(right)
60
61     if abs(f_left) <= epsilon:
62         return RootResult(interval, left, f_left, 0, 0.0, [])
63     if abs(f_right) <= epsilon:
64         return RootResult(interval, right, f_right, 0, 0.0, [])
65     if f_left * f_right > 0:
66         raise ValueError(
67             "На проміжку немає зміни знака функції, тому метод половинного "
68             "ділення не може бути застосований."
69         )
70
71     history: list[IterationRow] = []
72
73     for iteration in range(1, max_iterations + 1):

```



```

74     midpoint = (left + right) / 2.0
75     f_midpoint = func(midpoint)
76     interval_length = abs(right - left)
77     estimated_error = interval_length / 2.0
78
79     history.append(
80         IterationRow(
81             iteration=iteration,
82             left=left,
83             right=right,
84             midpoint=midpoint,
85             f_left=f_left,
86             f_midpoint=f_midpoint,
87             interval_length=interval_length,
88             estimated_error=estimated_error,
89         )
90     )
91
92     if abs(f_midpoint) <= epsilon or estimated_error <= epsilon:
93         return RootResult(
94             interval=interval,
95             root=midpoint,
96             function_value=f_midpoint,
97             iterations=iteration,
98             estimated_error=estimated_error,
99             history=history,
100         )
101
102     if f_left * f_midpoint < 0:
103         right = midpoint
104     else:
105         left = midpoint
106         f_left = f_midpoint
107
108     root = (left + right) / 2.0
109     return RootResult(
110         interval=interval,
111         root=root,
112         function_value=func(root),
113         iterations=max_iterations,
114         estimated_error=abs(right - left) / 2.0,
115         history=history,
116     )
117
118
119 def solve_all_intervals(
120     func: Function,
121     intervals: list[RootInterval],
122     epsilon: float,
123     max_iterations: int,
124 ) -> list[RootResult]:
125     return [
126         solve_bisection(func, interval, epsilon, max_iterations)
127         for interval in intervals

```

```

128     ]
129
130
131 def _deduplicate_intervals(
132     intervals: list[RootInterval],
133     zero_tolerance: float,
134 ) -> list[RootInterval]:
135     unique: list[RootInterval] = []
136
137     for interval in intervals:
138         is_duplicate = any(
139             abs(interval.left - item.left) <= zero_tolerance
140             and abs(interval.right - item.right) <= zero_tolerance
141             for item in unique
142         )
143         if not is_duplicate:
144             unique.append(interval)
145
146     return unique

```

(project/src/error_analysis.py)

```

1 from __future__ import annotations
2
3 from src.models import ErrorRow, RootResult
4
5
6 def build_error_rows(results: list[RootResult]) -> list[ErrorRow]:
7     return [
8         ErrorRow(
9             index=index,
10            interval=result.interval,
11            root=result.root,
12            function_value=result.function_value,
13            estimated_error=result.estimated_error,
14            iterations=result.iterations,
15        )
16        for index, result in enumerate(results, start=1)
17    ]
18
19
20 def max_residual(results: list[RootResult]) -> float:
21     if not results:
22         return 0.0
23     return max(abs(result.function_value) for result in results)
24
25
26 def max_estimated_error(results: list[RootResult]) -> float:
27     if not results:
28         return 0.0
29     return max(result.estimated_error for result in results)

```

(project/src/graph_build.py)

```

1  from __future__ import annotations
2
3  from collections.abc import Callable
4  from dataclasses import dataclass
5
6  from src.models import RootInterval, RootResult
7
8
9  Function = Callable[[float], float]
10
11
12  @dataclass(frozen=True)
13  class PlotData:
14      x_values: list[float]
15      y_values: list[float]
16      intervals: list[RootInterval]
17      results: list[RootResult]
18      left: float
19      right: float
20      y_min: float
21      y_max: float
22
23
24  def build_function_plot_data(
25      func: Function,
26      intervals: list[RootInterval],
27      results: list[RootResult],
28      left: float,
29      right: float,
30      plot_points: int,
31  ) -> PlotData:
32      if plot_points < 50:
33          raise ValueError("Кількість точок графіка має бути не меншою за 50.")
34
35      x_values = _build_x_values(left, right, plot_points)
36      y_values = [func(x_value) for x_value in x_values]
37      y_min = min(min(y_values), 0.0)
38      y_max = max(max(y_values), 0.0)
39
40      if y_min == y_max:
41          y_min -= 1.0
42          y_max += 1.0
43
44      padding = (y_max - y_min) * 0.08
45      return PlotData(
46          x_values=x_values,
47          y_values=y_values,
48          intervals=intervals,
49          results=results,
50          left=left,
51          right=right,
52          y_min=y_min - padding,
53          y_max=y_max + padding,

```

```

54     )
55
56
57 def _build_x_values(left: float, right: float, points: int) -> list[float]:
58     step = (right - left) / (points - 1)
59     return [left + index * step for index in range(points)]

```

(project/src/io_utils.py)

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5  from typing import Any
6
7
8  def load_json_config(path: str) -> dict[str, Any]:
9      with Path(path).open("r", encoding="utf-8") as file:
10         data = json.load(file)
11
12         if not isinstance(data, dict):
13             raise ValueError("JSON-конфігурація має бути об'єктом.")
14
15         return data
16
17
18 def save_text_report(path: str, report: str) -> None:
19     Path(path).write_text(report, encoding="utf-8")

```

(project/src/report.py)

```

1  from __future__ import annotations
2
3  from src.error_analysis import max_estimated_error, max_residual
4  from src.models import RootInterval, RootResult
5
6
7  def build_short_result(results: list[RootResult]) -> str:
8      if not results:
9          return "Корені не знайдено на заданому проміжку."
10
11     lines = ["Уточнені корені:"]
12     for index, result in enumerate(results, start=1):
13         lines.append(
14             f"{index}. x = {result.root:.10f}; "
15             f"f(x) = {result.function_value:.3e}; "
16             f"ітерацій = {result.iterations}"
17         )
18
19     lines.append(f"max |f(x)| = {max_residual(results):.3e}")
20     lines.append(f"max оцінка похибки = {max_estimated_error(results):.3e}")
21     return "\n".join(lines)

```

```

22
23
24 def build_full_report(
25     function_title: str,
26     left: float,
27     right: float,
28     scan_step: float,
29     epsilon: float,
30     max_iterations: int,
31     intervals: list[RootInterval],
32     results: list[RootResult],
33 ) -> str:
34     lines: list[str] = [
35         "Лабораторна робота N4",
36         "Тема: Розв'язання нелінійних рівнянь на комп'ютері",
37         "",
38         "Метод: метод половинного ділення.",
39         f"Рівняння: {function_title}",
40         "",
41         "Початкові дані:",
42         f"Проміжок пошуку: [{left:.10f}; {right:.10f}]",
43         f"Крок відокремлення коренів: {scan_step:.10f}",
44         f"Точність epsilon: {epsilon:.10g}",
45         f"Максимальна кількість ітерацій: {max_iterations}",
46         "",
47         "Відокремлені проміжки:",
48     ]
49
50     if intervals:
51         for index, interval in enumerate(intervals, start=1):
52             lines.append(f"{index}. {interval.as_text()}")
53     else:
54         lines.append("Корені на заданому проміжку не відокремлено.")
55
56     lines.extend(["", "Результати уточнення:"])
57     if results:
58         for index, result in enumerate(results, start=1):
59             lines.extend(
60                 [
61                     f"{index}. Проміжок: {result.interval.as_text()}",
62                     f"    Корінь: {result.root:.10f}",
63                     f"    f(x): {result.function_value:.12e}",
64                     f"    Оцінка похибки: {result.estimated_error:.12e}",
65                     f"    Кількість ітерацій: {result.iterations}",
66                 ]
67             )
68     else:
69         lines.append("Корені не уточнювалися, бо проміжки не знайдено.")
70
71     lines.extend(["", "Ітераційні таблиці:"])
72     for index, result in enumerate(results, start=1):
73         lines.append(f"Корінь {index}, проміжок {result.interval.as_text()}")
74         if not result.history:
75             lines.append("    Корінь збігається з межею проміжку.")

```

```

76         continue
77
78     lines.append(
79         "    k | a_k | b_k | x_k | f(a_k) | f(x_k) | довжина | похибка"
80     )
81     for row in result.history:
82         lines.append(
83             "    "
84             f"{row.iteration} | "
85             f"{row.left:.10f} | "
86             f"{row.right:.10f} | "
87             f"{row.midpoint:.10f} | "
88             f"{row.f_left:.6e} | "
89             f"{row.f_midpoint:.6e} | "
90             f"{row.interval_length:.6e} | "
91             f"{row.estimated_error:.6e}"
92         )
93
94     return "\n".join(lines)

```

(project/src/theme.py)

```

1  from __future__ import annotations
2
3
4  MAIN_THEME: dict[str, str] = {
5      "bg": "#0E0E0E",
6      "topbar": "#111111",
7      "panel": "#171717",
8      "surface": "#101010",
9      "surface_soft": "#2A2A2A",
10     "segment": "#242424",
11     "segment_hover": "#303030",
12     "segment_active": "#1F7A55",
13     "accent": "#36B37E",
14     "accent_hover": "#2F9F70",
15     "accent_text": "#07140E",
16     "text": "#F5F7FA",
17     "muted": "#AEB6BF",
18     "warning": "#F4D35E",
19     "error": "#F07167",
20 }

```

(project/src/gui.py)

```

1  from __future__ import annotations
2
3  import tkinter as tk
4  from dataclasses import dataclass
5  from tkinter import filedialog, ttk
6
7  import customtkinter as ctk

```

```

8
9 from src.bisection import isolate_root_intervals, solve_all_intervals
10 from src.error_analysis import build_error_rows
11 from src.functions import get_function_by_name
12 from src.graph_build import PlotData, build_function_plot_data
13 from src.io_utils import load_json_config, save_text_report
14 from src.models import PageConfig, RootInterval, RootResult
15 from src.report import build_full_report, build_short_result
16 from src.theme import MAIN_THEME
17
18
19 @dataclass
20 class BisectionState:
21     function_name: str = "target"
22     function_title: str = "2^x - 4x = 0"
23     left: float = 0.0
24     right: float = 4.0
25     scan_step: float = 0.1
26     epsilon: float = 1e-5
27     max_iterations: int = 100
28     plot_points: int = 500
29     intervals: list[RootInterval] | None = None
30     results: list[RootResult] | None = None
31     report: str = ""
32
33
34 class BasePage(ctk.CTkFrame):
35     def __init__(
36         self,
37         master: ctk.CTkFrame,
38         page_config: PageConfig,
39         fonts: dict[str, ctk.CTkFont],
40     ) -> None:
41         super().__init__(master, corner_radius=0)
42         self.page_config = page_config
43         self.fonts = fonts
44         self.grid_columnconfigure(0, weight=1)
45
46     def apply_theme(self, palette: dict[str, str]) -> None:
47         self.configure(fg_color=palette["bg"])
48
49
50 class MainPage(BasePage):
51     def __init__(
52         self,
53         master: ctk.CTkFrame,
54         page_config: PageConfig,
55         fonts: dict[str, ctk.CTkFont],
56         app_state: BisectionState,
57     ) -> None:
58         super().__init__(master, page_config, fonts)
59         self.app_state = app_state
60
61         self.result_var = tk.StringVar(value="Готово до обчислень")

```

```

62     self.error_var = tk.StringVar(value="")
63
64     self.grid_rowconfigure(3, weight=1)
65     self.grid_columnconfigure(0, weight=1)
66
67     self._build_header()
68     self._build_form()
69     self._build_actions()
70     self._build_output()
71     self.reset_defaults()
72
73     def _build_header(self) -> None:
74         self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
75         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
76         self.header_card.grid_columnconfigure(0, weight=1)
77
78         self.title_label = ctk.CTkLabel(
79             self.header_card,
80             text=self.page_config.title,
81             anchor="w",
82             font=self.fonts["title"],
83         )
84         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
85
86         self.subtitle_label = ctk.CTkLabel(
87             self.header_card,
88             text=self.page_config.subtitle,
89             anchor="w",
90             justify="left",
91             wraplength=860,
92             font=self.fonts["subtitle"],
93         )
94         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
95             ↪ 12))
96
97     def _build_form(self) -> None:
98         self.form_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
99         self.form_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
100         self.form_card.grid_columnconfigure((0, 1, 2), weight=1)
101
102         self.equation_label = ctk.CTkLabel(
103             self.form_card,
104             text="Рівняння:  $2^x - 4x = 0$ ",
105             anchor="w",
106             font=self.fonts["label"],
107         )
108         self.equation_label.grid(row=0, column=0, columnspan=2, sticky="ew",
109             ↪ padx=20, pady=(14, 6))
110
111         self.expected_label = ctk.CTkLabel(
112             self.form_card,
113             text="Очікувані корені: 0.310 та 4.000",
114             anchor="e",
115             font=self.fonts["label"],

```



```

114     )
115     self.expected_label.grid(row=0, column=2, sticky="ew", padx=(8, 20),
116                               ↪ pady=(14, 6))
117
118     self.entry_left = self._build_labeled_entry(self.form_card, 1, 0, "Ліва
119                               ↪ межа:")
120     self.entry_right = self._build_labeled_entry(self.form_card, 1, 1, "Права
121                               ↪ межа:")
122     self.entry_scan_step = self._build_labeled_entry(self.form_card, 1, 2,
123                               ↪ "Крок:")
124     self.entry_epsilon = self._build_labeled_entry(self.form_card, 3, 0,
125                               ↪ "epsilon:")
126     self.entry_max_iterations = self._build_labeled_entry(self.form_card, 3, 1,
127                               ↪ "Ітерації:")
128     self.entry_plot_points = self._build_labeled_entry(self.form_card, 3, 2,
129                               ↪ "Точки графіка:")
130
131     def _build_labeled_entry(
132         self,
133         parent: ctk.CTkFrame,
134         row: int,
135         column: int,
136         label_text: str,
137     ) -> ctk.CTkEntry:
138         label = ctk.CTkLabel(
139             parent,
140             text=label_text,
141             anchor="w",
142             font=self.fonts["label"],
143         )
144         label.grid(row=row, column=column, sticky="w", padx=20, pady=(0, 6))
145
146         entry = ctk.CTkEntry(
147             parent,
148             height=40,
149             corner_radius=8,
150             font=self.fonts["body"],
151         )
152         entry.grid(row=row + 1, column=column, sticky="ew", padx=20, pady=(0, 12))
153         return entry
154
155     def _build_actions(self) -> None:
156         self.actions = ctk.CTkFrame(self, fg_color="transparent")
157         self.actions.grid(row=2, column=0, sticky="ew", pady=(0, 10))
158         self.actions.grid_columnconfigure((0, 1, 2, 3), weight=1)
159
160         self.compute_button = ctk.CTkButton(
161             self.actions,
162             text="Обчислити",
163             height=40,
164             corner_radius=8,
165             font=self.fonts["button"],
166             command=self.compute,
167         )

```

```

161 self.compute_button.grid(row=0, column=0, sticky="ew", padx=(0, 7))
162
163 self.load_button = ctk.CTkButton(
164     self.actions,
165     text="Завантажити JSON",
166     height=40,
167     corner_radius=8,
168     font=self.fonts["button"],
169     command=self.load_json,
170 )
171 self.load_button.grid(row=0, column=1, sticky="ew", padx=7)
172
173 self.save_button = ctk.CTkButton(
174     self.actions,
175     text="Зберегти звіт",
176     height=40,
177     corner_radius=8,
178     font=self.fonts["button"],
179     command=self.save_report,
180 )
181 self.save_button.grid(row=0, column=2, sticky="ew", padx=7)
182
183 self.clear_button = ctk.CTkButton(
184     self.actions,
185     text="Очистити",
186     height=40,
187     corner_radius=8,
188     font=self.fonts["button"],
189     command=self.clear_all,
190 )
191 self.clear_button.grid(row=0, column=3, sticky="ew", padx=(7, 0))
192
193 def _build_output(self) -> None:
194     self.output_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
195     self.output_card.grid(row=3, column=0, sticky="nsew")
196     self.output_card.grid_columnconfigure(0, weight=1)
197     self.output_card.grid_rowconfigure(4, weight=1)
198
199     self.result_title = ctk.CTkLabel(
200         self.output_card,
201         text="Результат",
202         anchor="w",
203         font=self.fonts["label"],
204     )
205     self.result_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
206
207     self.result_label = ctk.CTkLabel(
208         self.output_card,
209         textvariable=self.result_var,
210         anchor="w",
211         justify="left",
212         wraplength=860,
213         font=self.fonts["result"],
214     )

```

```

215 self.result_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
216
217 self.error_label = ctk.CTkLabel(
218     self.output_card,
219     textvariable=self.error_var,
220     anchor="w",
221     justify="left",
222     wraplength=860,
223     font=self.fonts["body"],
224 )
225 self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 10))
226
227 self.table_title = ctk.CTkLabel(
228     self.output_card,
229     text="Таблиця коренів",
230     anchor="w",
231     font=self.fonts["label"],
232 )
233 self.table_title.grid(row=3, column=0, sticky="ew", padx=20, pady=(0, 6))
234
235 self.table_frame = tk.Frame(self.output_card, bg="#101010")
236 self.table_frame.grid(row=4, column=0, sticky="nsew", padx=20, pady=(0, 14))
237 self.table_frame.grid_rowconfigure(0, weight=1)
238 self.table_frame.grid_columnconfigure(0, weight=1)
239
240 columns = ("index", "interval", "root", "residual", "error", "iterations")
241 self.results_table = ttk.Treeview(
242     self.table_frame,
243     columns=columns,
244     show="headings",
245     height=11,
246 )
247
248 self.results_table.heading("index", text="N")
249 self.results_table.heading("interval", text="Проміжок")
250 self.results_table.heading("root", text="x")
251 self.results_table.heading("residual", text="f(x)")
252 self.results_table.heading("error", text="Помилка")
253 self.results_table.heading("iterations", text="Ітерації")
254
255 self.results_table.column("index", width=45, anchor="center")
256 self.results_table.column("interval", width=220, anchor="center")
257 self.results_table.column("root", width=140, anchor="center")
258 self.results_table.column("residual", width=130, anchor="center")
259 self.results_table.column("error", width=130, anchor="center")
260 self.results_table.column("iterations", width=80, anchor="center")
261
262 self.table_scrollbar = ttk.Scrollbar(
263     self.table_frame,
264     orient="vertical",
265     command=self.results_table.yview,
266 )
267 self.results_table.configure(yscrollcommand=self.table_scrollbar.set)
268

```

```

269         self.results_table.grid(row=0, column=0, sticky="nsew")
270         self.table_scrollbar.grid(row=0, column=1, sticky="ns")
271
272     def _fill_table(self, results: list[RootResult]) -> None:
273         for item in self.results_table.get_children():
274             self.results_table.delete(item)
275
276         for row in build_error_rows(results):
277             self.results_table.insert(
278                 "",
279                 "end",
280                 values=(
281                     row.index,
282                     row.interval.as_text(),
283                     f"{row.root:.10f}",
284                     f"{row.function_value:.3e}",
285                     f"{row.estimated_error:.3e}",
286                     row.iterations,
287                 ),
288             )
289
290     def load_json(self) -> None:
291         path = filedialog.askopenfilename(
292             title="Вибір JSON-файлу",
293             filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
294         )
295         if not path:
296             return
297
298         try:
299             loaded = load_json_config(path)
300
301             self._set_entry_value(self.entry_left, loaded.get("left", 0.0))
302             self._set_entry_value(self.entry_right, loaded.get("right", 4.0))
303             self._set_entry_value(self.entry_scan_step, loaded.get("scan_step",
304                 ↪ 0.1))
305             self._set_entry_value(self.entry_epsilon, loaded.get("epsilon", 1e-5))
306             self._set_entry_value(
307                 self.entry_max_iterations,
308                 loaded.get("max_iterations", 100),
309             )
310             self._set_entry_value(self.entry_plot_points, loaded.get("plot_points",
311                 ↪ 500))
312
313             self.result_var.set("JSON завантажено. Натисни «Обчислити».")
314             self.error_var.set("")
315         except Exception as exc:
316             self.result_var.set("Дані не завантажено")
317             self.error_var.set(f"Помилка: {exc}")
318
319     def _parse_input(self) -> tuple[float, float, float, float, int, int]:
320         left = float(self.entry_left.get().strip())
321         right = float(self.entry_right.get().strip())
322         scan_step = float(self.entry_scan_step.get().strip())

```

```

321     epsilon = float(self.entry_epsilon.get().strip())
322     max_iterations = int(self.entry_max_iterations.get().strip())
323     plot_points = int(self.entry_plot_points.get().strip())
324
325     if right <= left:
326         raise ValueError("Потрібно, щоб права межа була більшою за ліву.")
327     if scan_step <= 0:
328         raise ValueError("Крок має бути додатним.")
329     if scan_step > right - left:
330         raise ValueError("Крок не має перевищувати довжину проміжку пошуку.")
331     if epsilon <= 0:
332         raise ValueError("epsilon має бути додатним.")
333     if max_iterations < 1:
334         raise ValueError("Кількість ітерацій має бути не меншою за 1.")
335     if plot_points < 50:
336         raise ValueError("Кількість точок графіка має бути не меншою за 50.")
337
338     return left, right, scan_step, epsilon, max_iterations, plot_points
339
340 def compute(self) -> None:
341     try:
342         (
343             left,
344             right,
345             scan_step,
346             epsilon,
347             max_iterations,
348             plot_points,
349         ) = self._parse_input()
350         function_name = "target"
351         func, function_title = get_function_by_name(function_name)
352
353         intervals = isolate_root_intervals(func, left, right, scan_step)
354         results = solve_all_intervals(func, intervals, epsilon, max_iterations)
355         report = build_full_report(
356             function_title=function_title,
357             left=left,
358             right=right,
359             scan_step=scan_step,
360             epsilon=epsilon,
361             max_iterations=max_iterations,
362             intervals=intervals,
363             results=results,
364         )
365
366         self.app_state.function_name = function_name
367         self.app_state.function_title = function_title
368         self.app_state.left = left
369         self.app_state.right = right
370         self.app_state.scan_step = scan_step
371         self.app_state.epsilon = epsilon
372         self.app_state.max_iterations = max_iterations
373         self.app_state.plot_points = plot_points
374         self.app_state.intervals = intervals

```

```

375         self.app_state.results = results
376         self.app_state.report = report
377
378         self.result_var.set(build_short_result(results))
379         self.error_var.set("")
380         self._fill_table(results)
381     except Exception as exc:
382         self.result_var.set("Обчислення не виконано")
383         self.error_var.set(f"Помилка: {exc}")
384
385     def save_report(self) -> None:
386         if not self.app_state.report.strip():
387             self.result_var.set("Немає звіту")
388             self.error_var.set("Помилка: спочатку виконай обчислення.")
389             return
390
391         path = filedialog.asksaveasfilename(
392             title="Збереження звіту",
393             defaultextension=".txt",
394             filetypes=[("Text files", "*.txt"), ("All files", "*.*")],
395         )
396         if not path:
397             return
398
399         try:
400             save_text_report(path, self.app_state.report)
401             self.error_var.set("")
402         except Exception as exc:
403             self.error_var.set(f"Помилка: {exc}")
404
405     def clear_all(self) -> None:
406         self.reset_defaults()
407
408     def reset_defaults(self) -> None:
409         self._set_entry_value(self.entry_left, "0")
410         self._set_entry_value(self.entry_right, "4")
411         self._set_entry_value(self.entry_scan_step, "0.1")
412         self._set_entry_value(self.entry_epsilon, "0.00001")
413         self._set_entry_value(self.entry_max_iterations, "100")
414         self._set_entry_value(self.entry_plot_points, "500")
415
416         self.result_var.set("Готово до обчислень")
417         self.error_var.set("")
418
419         for item in self.results_table.get_children():
420             self.results_table.delete(item)
421
422         self.app_state.function_name = "target"
423         self.app_state.function_title = "2^x - 4x = 0"
424         self.app_state.left = 0.0
425         self.app_state.right = 4.0
426         self.app_state.scan_step = 0.1
427         self.app_state.epsilon = 1e-5
428         self.app_state.max_iterations = 100

```

```

429         self.app_state.plot_points = 500
430         self.app_state.intervals = None
431         self.app_state.results = None
432         self.app_state.report = ""
433
434     def _set_entry_value(self, entry: ctk.CTkEntry, value: object) -> None:
435         entry.delete(0, tk.END)
436         entry.insert(0, str(value))
437
438     def apply_theme(self, palette: dict[str, str]) -> None:
439         super().apply_theme(palette)
440
441         for card in (self.header_card, self.form_card, self.output_card):
442             card.configure(
443                 fg_color=palette["panel"],
444                 border_color=palette["surface_soft"],
445             )
446
447         self.title_label.configure(text_color=palette["text"])
448         self.subtitle_label.configure(text_color=palette["muted"])
449         self.equation_label.configure(text_color=palette["muted"])
450         self.expected_label.configure(text_color=palette["warning"])
451
452         for entry in (
453             self.entry_left,
454             self.entry_right,
455             self.entry_scan_step,
456             self.entry_epsilon,
457             self.entry_max_iterations,
458             self.entry_plot_points,
459         ):
460             entry.configure(
461                 fg_color=palette["surface"],
462                 border_color=palette["surface_soft"],
463                 text_color=palette["text"],
464             )
465
466         self.compute_button.configure(
467             fg_color=palette["accent"],
468             hover_color=palette["accent_hover"],
469             text_color=palette["accent_text"],
470         )
471         self.load_button.configure(
472             fg_color=palette["segment"],
473             hover_color=palette["segment_hover"],
474             text_color=palette["text"],
475         )
476         self.save_button.configure(
477             fg_color=palette["segment"],
478             hover_color=palette["segment_hover"],
479             text_color=palette["text"],
480         )
481         self.clear_button.configure(
482             fg_color=palette["segment"],

```

```

483         hover_color=palette["segment_hover"],
484         text_color=palette["text"],
485     )
486
487     self.result_title.configure(text_color=palette["muted"])
488     self.table_title.configure(text_color=palette["muted"])
489     self.result_label.configure(text_color=palette["text"])
490     self.error_label.configure(text_color=palette["error"])
491
492     style = ttk.Style()
493     style.theme_use("default")
494     style.configure(
495         "Treeview",
496         background="#101010",
497         foreground="#F5F7FA",
498         fieldbackground="#101010",
499         rowheight=28,
500         bordercolor="#2A2A2A",
501         borderwidth=1,
502     )
503     style.configure(
504         "Treeview.Heading",
505         background="#171717",
506         foreground="#F5F7FA",
507         relief="flat",
508     )
509     style.map(
510         "Treeview",
511         background=[("selected", "#1F7A55")],
512         foreground=[("selected", "#F5F7FA")],
513     )
514
515
516 class GraphPage(BasePage):
517     def __init__(
518         self,
519         master: ctk.CTkFrame,
520         page_config: PageConfig,
521         fonts: dict[str, ctk.CTkFont],
522         app_state: BisectionState,
523     ) -> None:
524         super().__init__(master, page_config, fonts)
525         self.app_state = app_state
526
527         self.status_var = tk.StringVar(value="Графік очікує на обчислення")
528         self.error_var = tk.StringVar(value="")
529         self.canvas: tk.Canvas | None = None
530         self.plot_data: PlotData | None = None
531         self.view_left = 0.0
532         self.view_right = 1.0
533         self.view_y_min = -1.0
534         self.view_y_max = 1.0
535         self.drag_start: tuple[int, int, float, float, float, float] | None = None
536

```



```

537         self.grid_rowconfigure(3, weight=1)
538
539     self._build_header()
540     self._build_controls()
541     self._build_output()
542     self._build_plot_area()
543
544     def _build_header(self) -> None:
545         self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
546         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
547         self.header_card.grid_columnconfigure(0, weight=1)
548
549         self.title_label = ctk.CTkLabel(
550             self.header_card,
551             text=self.page_config.title,
552             anchor="w",
553             font=self.fonts["title"],
554         )
555         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
556
557         self.subtitle_label = ctk.CTkLabel(
558             self.header_card,
559             text=self.page_config.subtitle,
560             anchor="w",
561             justify="left",
562             wraplength=860,
563             font=self.fonts["subtitle"],
564         )
565         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
566             ↪ 12))
567
568     def _build_controls(self) -> None:
569         self.controls_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
570         self.controls_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
571         self.controls_card.grid_columnconfigure((0, 1), weight=1)
572
573         self.info_label = ctk.CTkLabel(
574             self.controls_card,
575             text=(
576                 "Графік функції  $f(x)=2^x-4x$  з відокремленими проміжками "
577                 "та уточненими коренями. Перетягуй поле графіка мишею, "
578                 "коліщатко змінює масштаб."
579             ),
580             anchor="w",
581             justify="left",
582             wraplength=860,
583             font=self.fonts["body"],
584         )
585         self.info_label.grid(row=0, column=0, columnspan=2, sticky="ew", padx=20,
586             ↪ pady=(14, 12))
587
588         self.build_button = ctk.CTkButton(
589             self.controls_card,
590             text="Побудувати графік",

```

```

589         height=40,
590         corner_radius=8,
591         font=self.fonts["button"],
592         command=self.build_graph,
593     )
594     self.build_button.grid(row=1, column=0, sticky="ew", padx=(20, 8), pady=(0,
595 ↪ 14))
596
597     self.reset_view_button = ctk.CTkButton(
598         self.controls_card,
599         text="Скинути вид",
600         height=40,
601         corner_radius=8,
602         font=self.fonts["button"],
603         command=self.reset_plot_view,
604     )
605     self.reset_view_button.grid(row=1, column=1, sticky="ew", padx=(8, 20),
606 ↪ pady=(0, 14))
607
608     def _build_output(self) -> None:
609         self.output_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
610         self.output_card.grid(row=2, column=0, sticky="ew")
611         self.output_card.grid_columnconfigure(0, weight=1)
612
613         self.status_title = ctk.CTkLabel(
614             self.output_card,
615             text="Стан",
616             anchor="w",
617             font=self.fonts["label"],
618         )
619         self.status_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
620
621         self.status_label = ctk.CTkLabel(
622             self.output_card,
623             textvariable=self.status_var,
624             anchor="w",
625             justify="left",
626             wraplength=860,
627             font=self.fonts["result"],
628         )
629         self.status_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
630
631         self.error_label = ctk.CTkLabel(
632             self.output_card,
633             textvariable=self.error_var,
634             anchor="w",
635             justify="left",
636             wraplength=860,
637             font=self.fonts["body"],
638         )
639         self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 14))
640
641     def _build_plot_area(self) -> None:
642         self.plot_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)

```

```

641     self.plot_card.grid(row=3, column=0, sticky="nsew", pady=(14, 0))
642     self.plot_card.grid_columnconfigure(0, weight=1)
643     self.plot_card.grid_rowconfigure(0, weight=1)
644
645     self.plot_container = ctk.CTkFrame(self.plot_card, fg_color="transparent")
646     self.plot_container.grid(row=0, column=0, sticky="nsew", padx=12, pady=12)
647     self.plot_container.grid_columnconfigure(0, weight=1)
648     self.plot_container.grid_rowconfigure(0, weight=1)
649
650     def build_graph(self) -> None:
651         try:
652             if self.app_state.intervals is None or self.app_state.results is None:
653                 raise ValueError("Спочатку виконай обчислення на сторінці
654                                     ↪ «Обчислення».")
655
656             self.status_var.set("Побудова графіка...")
657             self.error_var.set("")
658             self.update_idletasks()
659
660             func, _ = get_function_by_name(self.app_state.function_name)
661             plot_data = build_function_plot_data(
662                 func=func,
663                 intervals=self.app_state.intervals,
664                 results=self.app_state.results,
665                 left=self.app_state.left,
666                 right=self.app_state.right,
667                 plot_points=self.app_state.plot_points,
668             )
669             self.plot_data = plot_data
670             self.reset_plot_view()
671
672             self.status_var.set("Графік побудовано")
673         except Exception as exc:
674             self.status_var.set("Графік не побудовано")
675             self.error_var.set(f"Помилка: {exc}")
676
677     def reset_plot_view(self) -> None:
678         if self.plot_data is None:
679             self.status_var.set("Немає графіка для скидання виду")
680             return
681
682         self.view_left = self.plot_data.left
683         self.view_right = self.plot_data.right
684         self.view_y_min = self.plot_data.y_min
685         self.view_y_max = self.plot_data.y_max
686         self.error_var.set("")
687         self.status_var.set("Вид скинуто")
688         self._draw_plot()
689
690     def _draw_plot(self) -> None:
691         if self.plot_data is None:
692             return
693
694         plot_data = self.plot_data

```

```

694     (
695         width,
696         height,
697         left_pad,
698         right_pad,
699         top_pad,
700         bottom_pad,
701         plot_width,
702         plot_height,
703     ) = self._plot_geometry()
704     palette = MAIN_THEME
705
706     if self.canvas is None:
707         self.canvas = tk.Canvas(
708             self.plot_container,
709             width=width,
710             height=height,
711             bg=palette["surface"],
712             highlightthickness=0,
713             cursor="hand2",
714         )
715         self.canvas.grid(row=0, column=0, sticky="nsew")
716         self.canvas.bind("<ButtonPress-1>", self._start_pan)
717         self.canvas.bind("<B1-Motion>", self._pan_plot)
718         self.canvas.bind("<ButtonRelease-1>", self._finish_pan)
719         self.canvas.bind("<MouseWheel>", self._zoom_plot)
720         self.canvas.bind("<Button-4>", self._zoom_plot)
721         self.canvas.bind("<Button-5>", self._zoom_plot)
722     else:
723         self.canvas.delete("all")
724
725     def map_x(x_value: float) -> float:
726         return left_pad + (x_value - self.view_left) / (
727             self.view_right - self.view_left
728         ) * plot_width
729
730     def map_y(y_value: float) -> float:
731         return top_pad + (self.view_y_max - y_value) / (
732             self.view_y_max - self.view_y_min
733         ) * plot_height
734
735     self.canvas.create_rectangle(
736         left_pad,
737         top_pad,
738         width - right_pad,
739         height - bottom_pad,
740         outline=palette["surface_soft"],
741         fill="#141414",
742     )
743
744     for index in range(6):
745         x_grid = left_pad + index * plot_width / 5
746         x_value = self.view_left + index * (self.view_right - self.view_left) /
             ↪ 5

```

```

747     self.canvas.create_line(
748         x_grid,
749         top_pad,
750         x_grid,
751         height - bottom_pad,
752         fill="#242424",
753     )
754     self.canvas.create_text(
755         x_grid,
756         height - 26,
757         text=f"{x_value:.2f}",
758         fill=palette["muted"],
759         font=("Segoe UI", 9),
760     )
761
762     y_grid = top_pad + index * plot_height / 5
763     y_value = self.view_y_max - index * (self.view_y_max - self.view_y_min)
764     ↪ / 5
765     self.canvas.create_line(
766         left_pad,
767         y_grid,
768         width - right_pad,
769         y_grid,
770         fill="#242424",
771     )
772     self.canvas.create_text(
773         34,
774         y_grid,
775         text=f"{y_value:.1f}",
776         fill=palette["muted"],
777         font=("Segoe UI", 9),
778     )
779
780     for interval in plot_data.intervals:
781         if interval.right < self.view_left or interval.left > self.view_right:
782             continue
783
784         x_start = map_x(interval.left)
785         x_end = map_x(interval.right)
786         if interval.width == 0:
787             if not self.view_left <= interval.left <= self.view_right:
788                 continue
789
790             self.canvas.create_line(
791                 x_start,
792                 top_pad,
793                 x_start,
794                 height - bottom_pad,
795                 fill=palette["error"],
796                 width=2,
797             )
798         else:
799             x_start = max(left_pad, min(width - right_pad, x_start))
800             x_end = max(left_pad, min(width - right_pad, x_end))

```

```

800         self.canvas.create_rectangle(
801             x_start,
802             top_pad,
803             x_end,
804             height - bottom_pad,
805             fill="#2F2A17",
806             outline="",
807         )
808
809     if self.view_y_min <= 0 <= self.view_y_max:
810         y_zero = map_y(0.0)
811         self.canvas.create_line(
812             left_pad,
813             y_zero,
814             width - right_pad,
815             y_zero,
816             fill=palette["warning"],
817             width=2,
818         )
819
820     if self.view_left <= 0 <= self.view_right:
821         x_zero = map_x(0.0)
822         self.canvas.create_line(
823             x_zero,
824             top_pad,
825             x_zero,
826             height - bottom_pad,
827             fill="#555555",
828             width=1,
829         )
830
831     coordinates: list[float] = []
832     for x_value, y_value in zip(plot_data.x_values, plot_data.y_values):
833         x_position = map_x(x_value)
834         y_position = map_y(y_value)
835         is_visible = (
836             left_pad <= x_position <= width - right_pad
837             and top_pad <= y_position <= height - bottom_pad
838         )
839
840         if is_visible:
841             coordinates.extend([x_position, y_position])
842         elif len(coordinates) >= 4:
843             self.canvas.create_line(
844                 *coordinates,
845                 fill=palette["accent"],
846                 width=2,
847                 smooth=True,
848             )
849             coordinates = []
850
851     if len(coordinates) >= 4:
852         self.canvas.create_line(
853             *coordinates,

```

```

854         fill=palette["accent"],
855         width=2,
856         smooth=True,
857     )
858
859     for result in plot_data.results:
860         if not (
861             self.view_left <= result.root <= self.view_right
862             and self.view_y_min <= 0 <= self.view_y_max
863         ):
864             continue
865
866         x_root = map_x(result.root)
867         y_root = map_y(0.0)
868         self.canvas.create_oval(
869             x_root - 5,
870             y_root - 5,
871             x_root + 5,
872             y_root + 5,
873             fill=palette["error"],
874             outline=palette["text"],
875             width=1,
876         )
877
878         self.canvas.create_text(
879             width / 2,
880             16,
881             text="f(x) = 2^x - 4x",
882             fill=palette["text"],
883             font=("Segoe UI", 11, "bold"),
884         )
885         self.canvas.create_text(
886             width / 2,
887             height - 10,
888             text="x",
889             fill=palette["muted"],
890             font=("Segoe UI", 10),
891         )
892         self.canvas.create_text(
893             16,
894             top_pad - 14,
895             text="f(x)",
896             fill=palette["muted"],
897             font=("Segoe UI", 10),
898         )
899
900     def _plot_geometry(self) -> tuple[int, int, int, int, int, int, int, int]:
901         width = 740
902         height = 500
903         left_pad = 62
904         right_pad = 28
905         top_pad = 34
906         bottom_pad = 48
907         plot_width = width - left_pad - right_pad

```

```

908     plot_height = height - top_pad - bottom_pad
909     return (
910         width,
911         height,
912         left_pad,
913         right_pad,
914         top_pad,
915         bottom_pad,
916         plot_width,
917         plot_height,
918     )
919
920 def _is_inside_plot(self, x_position: int, y_position: int) -> bool:
921     (
922         width,
923         height,
924         left_pad,
925         right_pad,
926         top_pad,
927         bottom_pad,
928         _plot_width,
929         _plot_height,
930     ) = self._plot_geometry()
931     return (
932         left_pad <= x_position <= width - right_pad
933         and top_pad <= y_position <= height - bottom_pad
934     )
935
936 def _start_pan(self, event: tk.Event) -> None:
937     if self.canvas is None or not self._is_inside_plot(event.x, event.y):
938         return
939
940     self.drag_start = (
941         event.x,
942         event.y,
943         self.view_left,
944         self.view_right,
945         self.view_y_min,
946         self.view_y_max,
947     )
948     self.canvas.configure(cursor="fleur")
949
950 def _pan_plot(self, event: tk.Event) -> None:
951     if self.drag_start is None:
952         return
953
954     (
955         _width,
956         _height,
957         _left_pad,
958         _right_pad,
959         _top_pad,
960         _bottom_pad,
961         plot_width,

```



```

962         plot_height,
963     ) = self._plot_geometry()
964     start_x, start_y, left, right, y_min, y_max = self.drag_start
965     x_shift = (event.x - start_x) / plot_width * (right - left)
966     y_shift = (event.y - start_y) / plot_height * (y_max - y_min)
967
968     self.view_left = left - x_shift
969     self.view_right = right - x_shift
970     self.view_y_min = y_min + y_shift
971     self.view_y_max = y_max + y_shift
972     self._draw_plot()
973
974     def _finish_pan(self, _event: tk.Event) -> None:
975         self.drag_start = None
976         if self.canvas is not None:
977             self.canvas.configure(cursor="hand2")
978
979     def _zoom_plot(self, event: tk.Event) -> str | None:
980         if self.plot_data is None or not self._is_inside_plot(event.x, event.y):
981             return None
982
983         (
984             _width,
985             _height,
986             left_pad,
987             _right_pad,
988             top_pad,
989             _bottom_pad,
990             plot_width,
991             plot_height,
992         ) = self._plot_geometry()
993         x_span = self.view_right - self.view_left
994         y_span = self.view_y_max - self.view_y_min
995
996         if x_span <= 0 or y_span <= 0:
997             return None
998
999         scroll_delta = getattr(event, "delta", 0)
1000         scroll_number = getattr(event, "num", None)
1001         zoom_in = scroll_delta > 0 or scroll_number == 4
1002         scale = 0.82 if zoom_in else 1.22
1003
1004         pointer_x_ratio = (event.x - left_pad) / plot_width
1005         pointer_y_ratio = (event.y - top_pad) / plot_height
1006         data_x = self.view_left + pointer_x_ratio * x_span
1007         data_y = self.view_y_max - pointer_y_ratio * y_span
1008         new_x_span = max(x_span * scale, 1e-8)
1009         new_y_span = max(y_span * scale, 1e-8)
1010
1011         self.view_left = data_x - pointer_x_ratio * new_x_span
1012         self.view_right = self.view_left + new_x_span
1013         self.view_y_max = data_y + pointer_y_ratio * new_y_span
1014         self.view_y_min = self.view_y_max - new_y_span
1015         self._draw_plot()

```

```

1016         return "break"
1017
1018     def apply_theme(self, palette: dict[str, str]) -> None:
1019         super().apply_theme(palette)
1020
1021         for card in (self.header_card, self.controls_card, self.output_card,
1022             ↪ self.plot_card):
1023             card.configure(
1024                 fg_color=palette["panel"],
1025                 border_color=palette["surface_soft"],
1026             )
1027
1028         self.title_label.configure(text_color=palette["text"])
1029         self.subtitle_label.configure(text_color=palette["muted"])
1030         self.info_label.configure(text_color=palette["muted"])
1031
1032         self.build_button.configure(
1033             fg_color=palette["accent"],
1034             hover_color=palette["accent_hover"],
1035             text_color=palette["accent_text"],
1036         )
1037         self.reset_view_button.configure(
1038             fg_color=palette["segment"],
1039             hover_color=palette["segment_hover"],
1040             text_color=palette["text"],
1041         )
1042
1043         self.status_title.configure(text_color=palette["muted"])
1044         self.status_label.configure(text_color=palette["text"])
1045         self.error_label.configure(text_color=palette["error"])
1046
1047     class AlgorithmsApp(CTk):
1048         def __init__(self) -> None:
1049             super().__init__()
1050             ctk.set_appearance_mode("dark")
1051
1052             self.title("BisectionMethodApp")
1053             self.resizable(False, False)
1054             self.geometry("820x900")
1055
1056             self.current_page = "main"
1057             self.nav_buttons: dict[str, CTkButton] = {}
1058             self.pages: dict[str, BasePage] = {}
1059             self.app_state = BisectionState()
1060
1061             self.fonts: dict[str, CTkFont] = {
1062                 "title": CTkFont(family="Segoe UI", size=27, weight="bold"),
1063                 "subtitle": CTkFont(family="Segoe UI", size=13),
1064                 "label": CTkFont(family="Segoe UI", size=13, weight="bold"),
1065                 "body": CTkFont(family="Segoe UI", size=12),
1066                 "button": CTkFont(family="Segoe UI", size=12, weight="bold"),
1067                 "result": CTkFont(family="Consolas", size=15, weight="bold"),
1068                 "nav": CTkFont(family="Segoe UI", size=12, weight="bold"),

```

```

1069     }
1070
1071     self.page_configs: tuple[PageConfig, ...] = (
1072         PageConfig(
1073             page_id="main",
1074             nav_title="Обчислення",
1075             title="Метод половинного ділення",
1076             subtitle=(
1077                 "Розв'язання рівняння  $2^x - 4x = 0$ . "
1078                 "Відокремлення коренів і уточнення з точністю epsilon."
1079             ),
1080         ),
1081         PageConfig(
1082             page_id="graph",
1083             nav_title="Графік",
1084             title="Графік функції",
1085             subtitle="Побудова  $f(x)$ , проміжків відокремлення та знайдених
1086                 ↪ коренів.",
1087         ),
1088     )
1089
1090     self.grid_columnconfigure(0, weight=1)
1091     self.grid_rowconfigure(1, weight=1)
1092
1093     self._build_topbar()
1094     self._build_pages()
1095     self._apply_theme()
1096     self.show_page(self.current_page)
1097
1098     def _build_topbar(self) -> None:
1099         self.topbar = ctk.CTkFrame(self, corner_radius=0)
1100         self.topbar.grid(row=0, column=0, sticky="ew")
1101         self.topbar.grid_columnconfigure(0, weight=1)
1102
1103         self.nav_frame = ctk.CTkFrame(self.topbar, fg_color="transparent")
1104         self.nav_frame.grid(row=0, column=0, sticky="w", padx=20, pady=(18, 12))
1105
1106         for index, page in enumerate(self.page_configs):
1107             button = ctk.CTkButton(
1108                 self.nav_frame,
1109                 text=page.nav_title,
1110                 width=130,
1111                 height=36,
1112                 corner_radius=8,
1113                 font=self.fonts["nav"],
1114                 border_width=1,
1115                 command=lambda pid=page.page_id: self.show_page(pid),
1116             )
1117             button.grid(row=0, column=index, padx=(0, 8))
1118             self.nav_buttons[page.page_id] = button
1119
1120     def _build_pages(self) -> None:
1121         self.page_host = ctk.CTkFrame(self, corner_radius=0)
1122         self.page_host.grid(row=1, column=0, sticky="nsew", padx=20, pady=(0, 20))

```

```

1122     self.page_host.grid_rowconfigure(0, weight=1)
1123     self.page_host.grid_columnconfigure(0, weight=1)
1124
1125     main_page = MainPage(
1126         self.page_host,
1127         self.page_configs[0],
1128         self.fonts,
1129         self.app_state,
1130     )
1131     main_page.grid(row=0, column=0, sticky="nsew")
1132     self.pages["main"] = main_page
1133
1134     graph_page = GraphPage(
1135         self.page_host,
1136         self.page_configs[1],
1137         self.fonts,
1138         self.app_state,
1139     )
1140     graph_page.grid(row=0, column=0, sticky="nsew")
1141     self.pages["graph"] = graph_page
1142
1143     def show_page(self, page_id: str) -> None:
1144         self.current_page = page_id
1145         self.pages[page_id].tkraise()
1146         self._refresh_nav_buttons()
1147
1148     def _refresh_nav_buttons(self) -> None:
1149         palette = MAIN_THEME
1150         for page_id, button in self.nav_buttons.items():
1151             if page_id == self.current_page:
1152                 button.configure(
1153                     fg_color=palette["segment_active"],
1154                     hover_color=palette["segment_active"],
1155                     text_color=palette["text"],
1156                     border_color=palette["segment_active"],
1157                 )
1158             else:
1159                 button.configure(
1160                     fg_color=palette["segment"],
1161                     hover_color=palette["segment_hover"],
1162                     text_color=palette["muted"],
1163                     border_color=palette["surface_soft"],
1164                 )
1165
1166     def _apply_theme(self) -> None:
1167         palette = MAIN_THEME
1168
1169         self.configure(fg_color=palette["bg"])
1170         self.topbar.configure(fg_color=palette["topbar"])
1171         self.page_host.configure(fg_color=palette["bg"])
1172
1173         for page in self.pages.values():
1174             page.apply_theme(palette)
1175

```

```
1176         self._refresh_nav_buttons()
1177
1178
1179 def main() -> None:
1180     app = AlgorithmsApp()
1181     app.mainloop()
```

Тестові JSON-файли:

Для перевірки завантаження конфігурації було підготовлено три JSON-файли. Стандартний файл має такий вигляд:

(project/test_json/default.json)

```
1 {
2     "left": 0.0,
3     "right": 4.0,
4     "scan_step": 0.1,
5     "epsilon": 0.00001,
6     "max_iterations": 100,
7     "plot_points": 500
8 }
```

Файл із вищою точністю:

(project/test_json/high_precision.json)

```
1 {
2     "left": 0.0,
3     "right": 4.0,
4     "scan_step": 0.05,
5     "epsilon": 0.00000001,
6     "max_iterations": 200,
7     "plot_points": 900
8 }
```

Файл із ширшим проміжком пошуку:

(project/test_json/wide_interval.json)

```
1 {
2     "left": -2.0,
3     "right": 6.0,
4     "scan_step": 0.2,
5     "epsilon": 0.00001,
6     "max_iterations": 120,
7     "plot_points": 800
8 }
```

Скріншоти демонстрації програми:

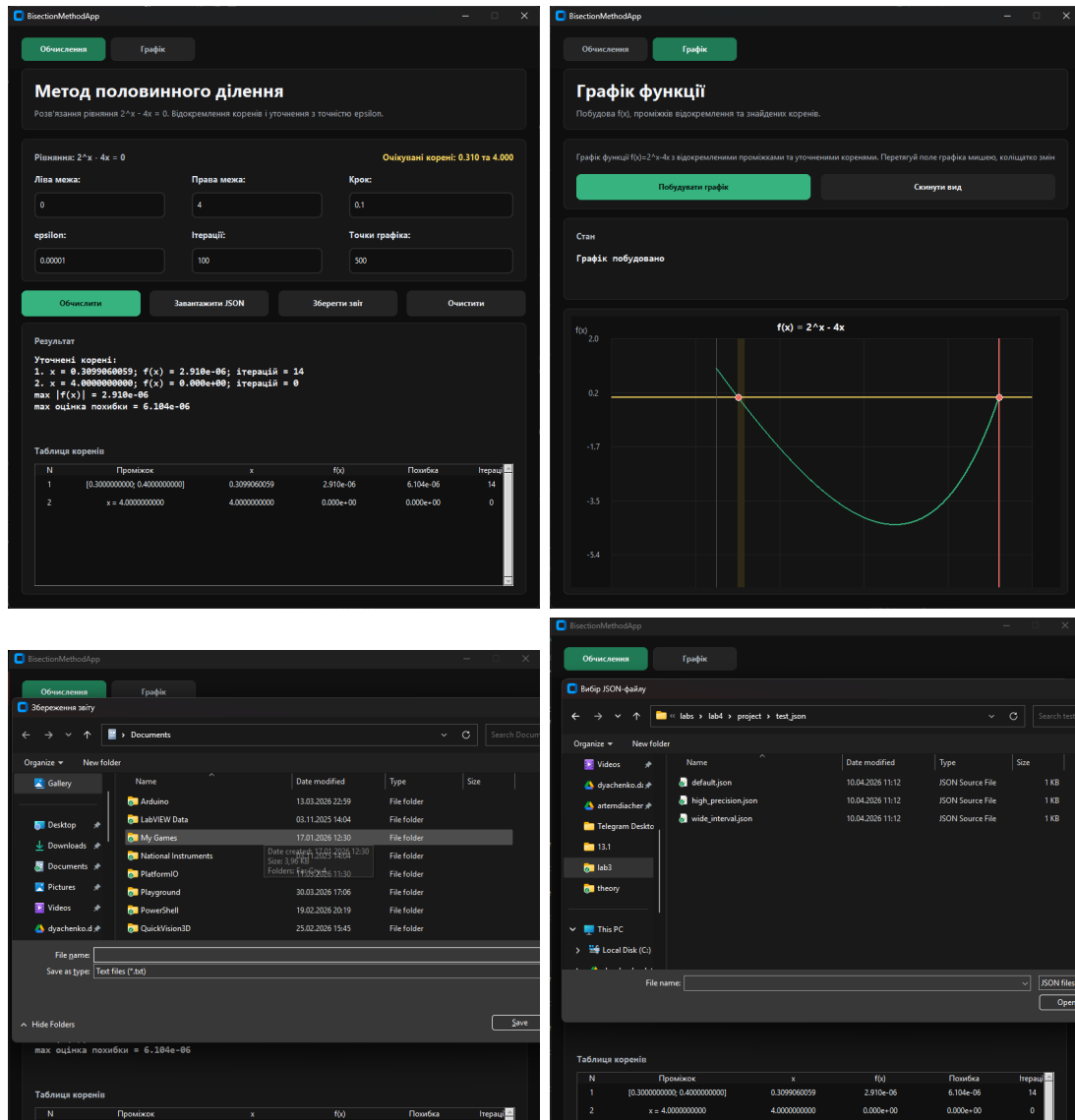


Рис. 2: Скріншоти демонстрації програми

Аналіз результатів

У ході виконання лабораторної роботи було реалізовано програму для розв'язання нелінійного рівняння

$$2^x - 4x = 0$$

методом половинного ділення. Спочатку було побудовано логіку відокремлення коренів шляхом сканування проміжку з фіксованим кроком. Після цього для кожного знайденого проміжку було застосовано метод половинного ділення, який послідовно звужує проміжок локалізації кореня.

Отримані результати показали, що рівняння має два корені на проміжку $[0; 4]$:

$$x_1 \approx 0.3099060059, \quad x_2 = 4.0000000000.$$

Для першого кореня було виконано 14 ітерацій, а значення нев'язки становить $2.909977 \cdot 10^{-6}$, що не перевищує заданої точності 10^{-5} . Другий корінь є точним, оскільки він збігається з правою межею проміжку і дає значення функції $f(4) = 0$.

Розроблена програма дозволяє вводити початкові дані вручну або завантажувати їх із JSON-файлу, переглядати результати у таблиці, зберігати звіт та аналізувати функцію за допомогою інтерактивного графіка. Це підтверджує коректність реалізації алгоритму та практичну придатність програми для дослідження нелінійних рівнянь.